Question 1 – Morphological Analysis Error in a Biomedical Dataset

1. Aim

To identify errors caused by Porter Stemming in biomedical text and understand inflectional and derivational morphology.

2. Python program

```
import pandas as pd
from nltk.stem import PorterStemmer

ps = PorterStemmer()

words = [
    "infection", "infectious", "infected",
    "infect", "infections", "infecting"
]

data = pd.DataFrame({"Original": words})
data["Stem"] = data["Original"].apply(ps.stem)

print(data)
```

```
     Original     Stem
0   infection   infect
1  infectious  infecti
2    infected   infect
3      infect   infect
4  infections   infect
5   infecting   infect
```

3. Error analysis

Word	Morphology	Explanation
infection	Derivational	Noun derived from infect
infectious	Derivational	Adjective with a distinct meaning
infected	Inflectional	Past-tense or participle form
infections	Inflectional	Plural noun
infecting	Inflectional	Present participle

Porter Stemmer may produce different stems for related words, such as infecti for infectious and infect for infected.

4. Inflectional vs. derivational morphology

- Inflectional: Changes grammatical form, such as infect → infected.
- Derivational: Creates a new word or changes its grammatical category, such as infect → infection.

5. Conclusion

Stemming can reduce search accuracy when medically meaningful word forms are merged incorrectly. Lemmatization and biomedical vocabulary rules can improve retrieval quality.

Question 2 – Error Analysis of a Finite-State Morphological Parser

1. Aim

To correct a parser that fails to recognize multiple prefixes and suffixes.

2. Correct morphological analysis

Word	Analysis
happiest	happy + -est
unbelievable	un- + believe + -able
running	run + -ing
reordering	re- + order + -ing
smartphones	smartphone + -s
unreadable	un- + read + -able

3. Corrected Python program

```
def parser(word):
    rules = {
        "happiest": ("happy", "-est"),
        "unbelievable": ("believe", "un- + -able"),
        "running": ("run", "-ing"),
        "reordering": ("order", "re- + -ing"),
        "smartphones": ("smartphone", "-s"),
        "unreadable": ("read", "un- + -able")
    }

    if word in rules:
        return rules[word]
    return word, "No rule found"
words = [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]
for word in words:
    print(word, ":", parser(word))
```

```
happiest : ('happy', '-est')
unbelievable : ('believe', 'un- + -able')
running : ('run', '-ing')
reordering : ('order', 're- + -ing')
smartphones : ('smartphone', '-s')
unreadable : ('read', 'un- + -able')
```

4. Why the original parser fails

- It recognizes only limited affix patterns.
- It cannot handle multiple affixes.

- It ignores spelling changes, such as happy → happiest.
- It may fail on unfamiliar or irregular words.

5. Evaluation and complexity

Accuracy is calculated as:

$$\text{Accuracy}=\frac{\text{Correct analyses}}{\text{Total words}}\times100$$

If all six test words are analyzed correctly, accuracy is 100%. The corrected example uses explicit dictionary rules rather than a general FST. For a fixed number of rules, matching and lookup are efficient; a full FST needs additional transition and ambiguity handling.

6. Conclusion

Combining prefix rules, suffix rules, spelling changes, and a dictionary improves morphological analysis.

Question 3 – Error Analysis of Stemming in News Classification

1. Aim

To compare classification using original text, Porter Stemming, and lemmatization.

2. Stemming errors

Word	Possible stem	Problem
organization	organ	Meaning may be lost
organizer	organ	Different words are merged
organizing	organ	Action meaning is lost
organized	organ	Grammatical information is lost

3. Python program

```
import re
import time
import pandas as pd
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.datasets import fetch_20newsgroups
nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)
newsgroups_data = fetch_20newsgroups(subset='all', categories=['sci.space', 'rec.sport.baseball', 'talk.politics.mideast'], remove=
df = pd.DataFrame({'Text': newsgroups_data.data, 'Category': newsgroups_data.target})
df = df.dropna(subset=["Text", "Category"]).copy()
df = df[df["Text"].str.strip() != ""]
df["Text"] = df["Text"].astype(str)
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
def process(text, method):
    words = re.findall(r"[a-z]+", text.lower())
    if method == "porter":
        words = [stemmer.stem(word) for word in words]
    elif method == "lemma":
        words = [lemmatizer.lemmatize(word) for word in words]
    return " ".join(words)
X_train, X_test, y_train, y_test = train_test_split(
    df["Text"],
```

```
...
    Classification Results
    Method  Vocabulary Size  Accuracy  Processing Time (seconds)
      none            26594     92.95                      2.886
    porter            18664     93.12                     10.488
     lemma            23797     92.95                      4.046
```

4. Comparison

Method	Advantage	Limitation
No stemming	Preserves word forms	Larger vocabulary
Porter stemming	Reduces vocabulary	May lose meaning
Lemmatization	Produces meaningful base forms	Requires more processing

5. Conclusion

Lemmatization may preserve more semantic information, but the best method must be selected using actual classification accuracy and confusion-matrix results.

Question 4 – Morphological Error Analysis in Information Retrieval

1. Aim

To study how suffix removal affects product search relevance.

2. Error analysis

Word	Type	Explanation
watches	Inflectional	Plural of watch
watching	Inflectional	Verb form
washable	Derivational	Means capable of being washed
washer	Derivational	Person or object that washes
washed	Inflectional	Past-tense or participle form

3. Improved Python program

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
words = ["watches", "watching", "washable",
         "washer", "washed"]
for word in words:
    print(word, "->",
          lemmatizer.lemmatize(word, pos="v"))
```

```
watches -> watch
watching -> watch
washable -> washable
washer -> washer
washed -> wash
```

4. Improved strategy

1. Convert text to lowercase.
2. Use lemmatization for grammatical variations.
3. Preserve meaningful derivational suffixes.
4. Use product categories and attributes.
5. Evaluate search relevance using precision and recall.

5. Conclusion

Context-aware normalization helps prevent irrelevant product recommendations caused by excessive stemming.

Question 5 – Identify and Correct the Coding Errors

1. Errors

The original code applies ps.stem() to an entire document instead of individual words. It also assumes that the CSV file exists and contains a Text column.

2. Corrected program

import pandas as pdimport refrom nltk.stem import PorterStemmerps = PorterStemmer()data = pd.read_csv("BBCNews.csv")def stem_text(text): if pd.isna(text): return "" words = re.findall(r"[a-zA-Z]+", str(text).lower()) return " ".join(ps.stem(word) for word in words)data["Processed"] = data["Text"].apply(stem_text)print(data[["Text", "Processed"]].head())data.to_csv("BBCNews_stemmed.csv", index=False)

3. Example output

Original word	**Stem**
running	run
studies	studi
computers	comput
organization	organ
organized	organ

4. Twenty cases for error analysis

Word	**Concern**	**Morphology**
organization	Meaning may be lost	Derivational
organizer	Different word meaning	Derivational
organizing	Action meaning	Inflectional
organized	Tense information	Inflectional
studies	Noun/verb distinction	Inflectional
studying	Verb form	Inflectional
happier	Comparative meaning	Inflectional
happiness	Abstract noun meaning	Derivational
relational	Adjective meaning	Derivational

national	Adjective meaning	Derivational
runner	Agent meaning	Derivational
running	Verb form	Inflectional
washable	Capability meaning	Derivational
washer	Object meaning	Derivational
washed	Tense information	Inflectional
infectious	Medical adjective meaning	Derivational
infection	Medical noun meaning	Derivational
infected	Grammatical form	Inflectional
unbelievable	Negation and adjective meaning	Derivational
unreadable	Negation and adjective meaning	Derivational

Note: These are potential information-loss cases, not all confirmed Porter Stemmer errors. Run the program to verify exact stems.

5. Conclusion

Tokenizing before stemming corrects the program. Comparing original and processed words helps identify harmful normalization.

Question 6 – Error Analysis of a Plural Noun Parser

1. Errors in the original code

The statement word[:-2] removes two characters instead of one for regular plurals. It also fails to handle -ies, -es, and irregular plurals.

2. Corrected program

```
def parser(word):
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "mice": "mouse",
        "feet": "foot"
    }
    if word in irregular:
        return irregular[word], "Plural"
    if word.endswith("ies"):
        return word[:-3] + "y", "Plural"
    if word.endswith(("ches", "shes", "xes", "zes", "sses")):
        return word[:-2], "Plural"
    if word.endswith("s") and not word.endswith(("ss", "us", "is")):
        return word[:-1], "Plural"
    return word, "Singular or Unknown"
for word in ["cars", "boxes", "cities", "children"]:
    print(word, "->", parser(word))
```

```
cars -> ('car', 'Plural')
boxes -> ('box', 'Plural')
cities -> ('city', 'Plural')
children -> ('child', 'Plural')
```

3. Expected output

cars -> ('car', 'Plural')

boxes -> ('box', 'Plural')

cities -> ('city', 'Plural')

children -> ('child', 'Plural')

4. Limitations

- Some words have irregular plural forms.
- Words ending in s are not always plural.
- Spelling rules cannot handle every English word.
- A dictionary such as WordNet improves validation.

5. Conclusion

Combining rules and a lexical database produces more reliable plural analysis than simply removing characters.

Question 7 – Morphological Feature Extraction Error

1. Identify the error

The original code performs vectorization before stemming. Stemming the feature names afterward does not merge the original columns in the feature matrix.

2. Corrected program

```
import re
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
stemmer = PorterStemmer()
documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]
def preprocess(text):
    words = re.findall(r"[a-zA-Z]+", text.lower())
    return " ".join(stemmer.stem(w) for w in words)
processed = [preprocess(doc) for doc in documents]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(processed)
print("Processed documents:", processed)
print("Vocabulary:", vectorizer.get_feature_names_out())
print("Vocabulary size:", len(vectorizer.vocabulary_))
print(X.toarray())
```

```
Processed documents: ['run runner run', 'studi studi studi', 'organ organ organ']
Vocabulary: ['organ' 'run' 'runner' 'studi']
Vocabulary size: 4
[[0 2 1 0]
 [0 0 0 3]
 [3 0 0 0]]
```

3. Expected observations

- Related word forms may share a feature.
- Vocabulary size may decrease.
- The feature matrix now uses stemmed tokens consistently.
- Classification accuracy may improve or decrease depending on the dataset.

4. Comparison

Metric	Before correction	After correction
Vocabulary	Original word forms	Stemmed word forms
Vocabulary size	Often larger	Often smaller
Classification accuracy	Baseline	Must be measured
Processing time	Lower preprocessing cost	Additional stemming time

For the 20 Newsgroups dataset, the same train-test split and classifier should be used for both methods to ensure a fair comparison.

5. Conclusion

Stemming must be applied before vectorization so the feature matrix uses normalized terms. The improvement should be confirmed by measuring vocabulary size, classification accuracy, and processing time.

Overall conclusion

Morphological error analysis helps identify problems in stemming, finite-state parsing, and feature extraction. Porter Stemming is fast but can remove meaningful information. Lemmatization, lexical validation, and correctly ordered preprocessing can improve NLP performance when evaluated on real datasets.

See another version